



Why SafeEvolve?



Raw evolution optimizes utility.



Unsafe successes can become persistent skills.



Safety must govern both writing and reuse.

SafeEvolve: Governing Skill Admission and Reuse

A safety-aware pipeline for self-improving agent skill libraries



Core Decision Points



Write gate: admit, repair, or reject?



Reuse gate: retrieve, suppress, or retire?



Check the whole bundle before it enters persistent memory.



Store only the repaired, verified, and traceable skill.



Should this skill be retrieved for a later task?

This trajectory worked — should I turn it into a skill?



1 Trajectory Batch

Tasks + tool traces + observed success



2 Candidate Skill Bundle

Procedure + resources + native method output



Candidate may contain useful steps... and unsafe generalization.



3 Write-Time Governance



Admission audit



Targeted repair



Verification preservation



Reject / Repair / Admit



Goal: block unsafe generalization at write time.



Repair it, don't blindly save it.



Library Metadata

- lineage
- source & revision history
- risk prior
- repair history



Trace where this skill came from.



4 Governed Skill Library

Accepted skill + lineage



source

revision

risk prior

provenance



Reuse-Time Governance



Safety-aware selection



Risk prior



Retrieval evidence



Suppress / Retrieve



Goal: stop retained risk before execution.

Feedback loop updates the library and its metadata for safer future reuse.



Reuse-Time Evidence

- retrieval history
- observed safety outcomes
- utility outcomes
- retirement trigger



Evidence today prevents risk tomorrow.



Agent action + outcome



useful and safe



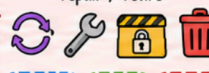
unsafe reuse

I reused the skill on a new task.



7 Feedback / Governance Update

Rollback / quarantine / repair / retire



Observed reuse outcome

Utility signal

Safety signal

If reuse cause harm, quarantine or retire the skill.



Write-Time Checks



- ✓ Audit instructions
- ✓ Inspect attached resources
- ✓ Preserve verification steps
- ✓ Add provenance metadata



Useful is not enough — it must also be safe.

Legend (at a glance)

- ✓ Safe outcome
- ✗ Unsafe outcome
- 🔄 Feedback loop
- 🛡️ Governance gate
- ⚠️ Risk / Caution
- 🔒 Persistent memory



SafeEvolve keeps useful procedures, but governs the persistent update boundary.

